DV INTERVIEW · GLANCE SHEET

AXI Protocol: Channels, Handshake & Ordering

AMBA Advanced eXtensible Interface — 5 independent channels, burst-based, ID-tagged, multiple outstanding transactions.

`Topic: Intro to AXI · AXI channels · In-order vs out-of-order transactions`

`Golden rule → transfer happens only when VALID & READY are both HIGH at a rising clock edge`

1 Where AXI sits in AMBA

Bus	Use	Traits
APB	Slow peripherals (UART, GPIO, timers)	No pipelining, no burst, simple setup → access
AHB	Mid-performance system bus	Single shared channel, pipelined addr/data, one master at a time
AXI	High-performance SoC fabric (CPU, DDR, DMA, GPU)	5 separate channels, bursts, IDs, outstanding & out-of-order
AXI-Lite	Register / control access	Single-beat only, no IDs, no bursts
AXI-Stream	Data pipes (video, DSP, network)	No address; TVALID/TREADY/TDATA/TLAST

Terminology: **Master / Manager** initiates, **Slave / Subordinate** responds. An interconnect connects many of each.

2 Why AXI is fast

- **Separate address/control and data phases** — address can be sent ahead of data.
- **Separate read and write channels** — reads and writes run simultaneously (full duplex).
- **Burst transactions** — only the *start address* is sent; slave computes the rest.
- **Multiple outstanding transactions** — issue new addresses before earlier ones complete.
- **Out-of-order completion** using transaction IDs.
- **Unaligned transfers & byte lanes** via `WSTRB`.
- **Register slices** can be inserted on any channel for timing closure (channels are independent).

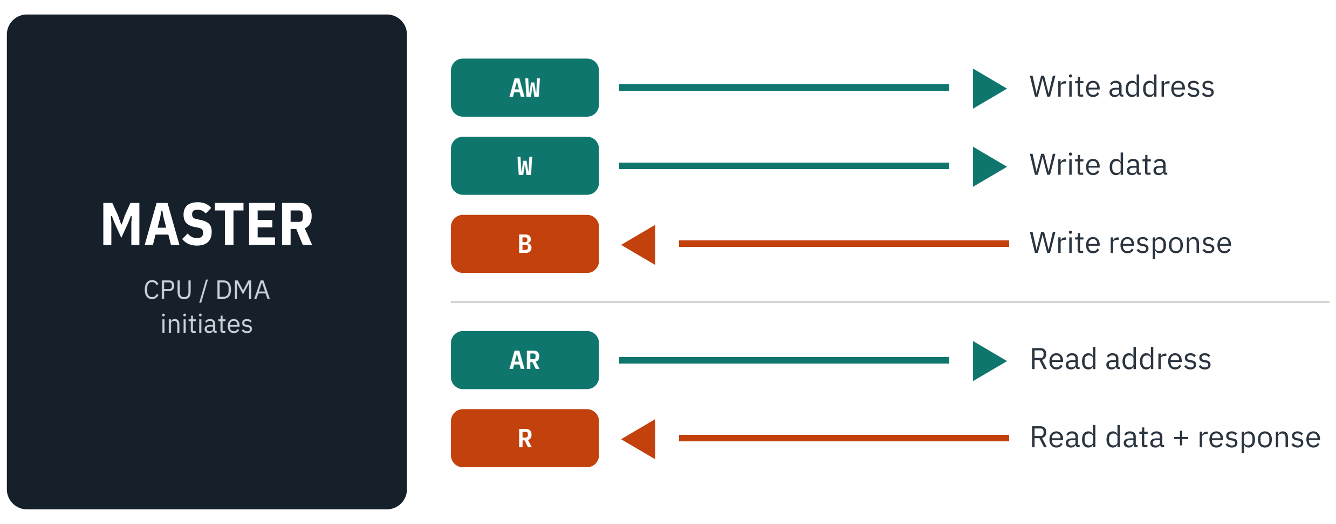
One-liner: AXI decouples address, data and response so latency is hidden by pipelining and parallelism.

3 AXI3 vs AXI4 (asked a lot)

Feature	AXI3	AXI4
INCR burst length	1–16	**1–256**
AxLEN width	4 bits	8 bits
Write interleaving	Allowed (has `WID`)	**Not allowed** — WID removed
Locked transfers	Locked + exclusive	Exclusive only (AxLOCK 1 bit)
New signals	—	`AxQOS`, `AxREGION`, `xUSER`
WRAP / FIXED length	WRAP 2/4/8/16; FIXED ≤16	Same (only INCR grew)

AXI4 write response may only come after **both** AW and the last W beat are handshaken.

4 The 5 channels — each has its own VALID / READY



MASTER
CPU / DMA initiates

AW → Write address
W → Write data
B ← Write response
AR → Read address
R ← Read data + response

SLAVE
Memory / peripheral responds

Ch	Key signals	Note
AW	`AWID AWADDR AWLEN AWSIZE AWBURST`	One per burst
W	`WDATA WSTRB WLAST`	One per beat; WLAST on final beat
B	`BID BRESP`	One per *burst*, not per beat
AR	`ARID ARADDR ARLEN ARSIZE ARBURST`	One per burst
R	`RID RDATA RRESP RLAST`	RRESP on *every* beat

Write = 3 channels (AW, W, B). **Read = 2 channels** (AR, R) — read response rides on the R channel, so there's no separate read-response channel.

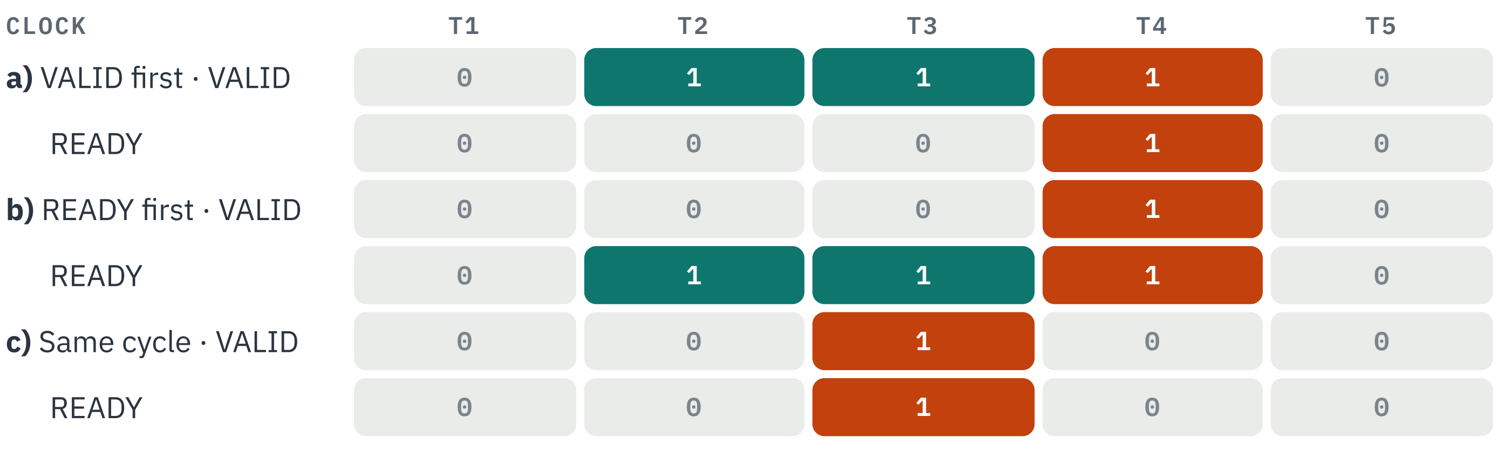
Direction: AW, W, AR driven by master; B and R driven by slave. READY always flows opposite to VALID.

Why no separate read response? Data already returns slave → master, so RRESP is attached to each R beat for free.

5 VALID / READY handshake

- **Source** drives VALID (info available); **destination** drives READY (can accept).
- Transfer occurs on the rising edge where **VALID = 1 and READY = 1**.
- **Once VALID is asserted it must stay HIGH**, with data/control stable, until the handshake completes.
- **Source must NOT wait for READY** before asserting VALID (else deadlock).
- **Destination MAY wait for VALID** before asserting READY; READY may also be asserted early and de-asserted before VALID.

THREE LEGAL ORDERINGS (TRANSFER CYCLE IN ORANGE)



CLOCK	T1	T2	T3	T4	T5
a) VALID first · VALID	0	1	1	1	0
READY	0	0	0	1	0
b) READY first · VALID	0	0	0	1	0
READY	0	1	1	1	0
c) Same cycle · VALID	0	0	1	0	0
READY	0	0	1	0	0

Interview trap: "Can VALID drop before READY comes?" → No, that's a protocol violation. "Can READY drop without VALID?" → Yes.

6 Channel dependencies

WRITE TRANSACTION

`AW handshake` + `W … WLAST handshake` → `B response`

- W data **may arrive before** AW (slave may wait for both).
- Slave must wait for **WLAST** (and in AXI4, AW too) before asserting BVALID.

READ TRANSACTION

`AR handshake` → `R beats … RLAST`

- Slave must wait for **ARVALID & ARREADY** before asserting RVALID.

DEADLOCK-AVOIDANCE RULES

- Slave may wait for AWVALID/WVALID before AWREADY/WREADY.
- Master must **not** wait for AWREADY/WREADY before driving W/AW VALID.
- Master may wait for BVALID before BREADY (but it's good practice to keep BREADY/RREADY high).

Count rule: #W beats per burst = AWLEN + 1, WLAST exactly on the last one. #R beats = ARLEN + 1, RLAST on the last.

7 Bursts: LEN · SIZE · TYPE

AxLEN
beats – 1
`beats = LEN + 1`

AxSIZE
bytes per beat
`bytes = 2^SIZE` (≤ bus width)

Total bytes
`(LEN+1) × 2^SIZE`

AxBURST	Type	Address behaviour	Typical use
`2'b00`	**FIXED**	Same address every beat	FIFO / peripheral register
`2'b01`	**INCR**	Address += 2^SIZE each beat	Normal memory access
`2'b10`	**WRAP**	Increments, then wraps at boundary	Cache-line fill (critical word first)
`2'b11`	Reserved	—	Flag as error in checker

4 KB rule: a burst must never cross a 4 KB address boundary (a slave region is ≥ 4 KB, so a burst stays in one slave).

WRAP rules: length 2, 4, 8 or 16 only; start address must be aligned to transfer size.

WORKED WRAP EXAMPLE

WRAP, 4 beats, 4 bytes/beat, start `0x38` → container = 4 × 4 = 16 B → lower boundary = `INT(0x38/16)×16 = 0x30`, upper = 0x40

`0x38` → `0x3C` → wrap → `0x30` → `0x34`

Same with **INCR**: 0x38 → 0x3C → 0x40 → 0x44. With **FIXED**: 0x38 ×4.

8 Responses & side-band signals

xRESP	Name	Meaning
`00`	**OKAY**	Normal success (or exclusive access failed)
`01`	**EXOKAY**	Exclusive access succeeded
`10`	**SLVERR**	Slave reached, but it reported an error
`11`	**DECERR**	Decode error — no slave at that address (interconnect default slave)

Signal	Purpose
`WSTRB`	1 bit per data byte; which lanes are valid (narrow/unaligned writes)
`AxLOCK`	Exclusive access (atomic read-modify-write semaphores)
`AxCACHE`	Bufferable, modifiable, allocate hints
`AxPROT`	Privileged / secure / instruction vs data
`AxQOS`	Priority hint for arbitration (AXI4)
`AxREGION`	Multiple logical regions in one slave (AXI4)
`xUSER`	User-defined side-band (AXI4)

WSTRB example (32-bit bus): write 1 byte at 0x1002 → `WSTRB = 4'b0100`. Narrow transfer: SIZE < bus width, active lanes rotate each beat.

9 Transaction IDs: in-order vs out-of-order

THE RULE

Same ID → must complete in issue order.
Different IDs → may complete in any order.

- ID = "ordering tag", typically one per master thread / stream.
- The interconnect **extends the ID** with master-port bits so responses route back to the right master.
- No ordering between reads and writes, even with the same ID — master must wait for a response if it needs that.

EXAMPLE — MASTER ISSUES 3 READS

Issued: `A · ID0` `B · ID1` `C · ID0`
Legal: `B` `A` `C` ✓ B overtakes (diff ID)
Legal: `A` `B` `C` ✓ in-order
Illegal: `C` `A` `B` × C before A (same ID0)

Why out-of-order? A fast slave (SRAM) shouldn't be blocked by a slow slave (DDR). Different IDs let the fast response return first.

OUTSTANDING & INTERLEAVING

- **Outstanding:** master sends next AR/AW before previous data/response returns. Depth = how many in flight (read/write acceptance capability).
- **Read data interleaving:** beats of bursts with *different* RIDs may be mixed; within one burst, beats stay in order.
- **Write data interleaving:** AXI3 only (WID). **AXI4: W beats must follow AW order**, no interleaving.
- In-order slaves simply return everything in issue order — always legal.

Mode	Pros	Cons
In-order	Simple slave & master	Head-of-line blocking
Out-of-order	Higher throughput, lower latency	Reorder buffers, ID tracking

10 How a DV engineer verifies AXI

TESTBENCH (UVM AXI VIP)

- Agent per interface: **sequencer · driver · monitor**; separate drivers/threads per channel so channels stay independent.
- Monitor rebuilds bursts from beats; sends to scoreboard via analysis port.
- **Scoreboard:** reference memory model + **per-ID queues** (assoc array keyed by ID) → checks same-ID order, allows cross-ID reordering.
- Randomize READY delays (back-pressure) and VALID delays.

SVA PROTOCOL CHECKS

- VALID stays high & payload stable until READY: `valid && !ready |=> valid && $stable(payload)`
- No X on VALID after reset; VALIDs low during reset.
- WLAST/RLAST on beat LEN+1 exactly; no beat after LAST.
- No 4 KB crossing; WRAP len ∈ {2,4,8,16}; WRAP addr aligned; BURST ≠ 2'b11.
- BVALID only after AW + WLAST; RVALID only after AR; BID/RID match an outstanding ID.

FUNCTIONAL COVERAGE

- Cross **BURST × LEN × SIZE**, aligned vs unaligned, all WSTRB patterns.
- All 4 responses, outstanding depth (0 … max), same-ID vs diff-ID reorder seen.
- Handshake orderings a/b/c, back-to-back transfers, reads & writes concurrently.

11 Rapid-fire interview Q&A

Question	Crisp answer
How many channels? Why separate?	5 (AW, W, B, AR, R); independent = parallel R/W, pipelining, register slices.
Why no read-response channel?	RRESP travels with each R beat.
Max burst length?	AXI4 INCR 256; AXI3 16; WRAP 16; FIXED 16.
Why the 4 KB boundary?	Min slave address region; keeps a burst inside one slave & one page.
Can W come before AW?	Yes; slave may buffer it.
Purpose of WLAST if LEN is known?	Slave needn't count beats; simplifies logic & checking.
Why was WID removed in AXI4?	Write interleaving was rarely used and complicated slaves.
How is out-of-order achieved?	Different IDs; same-ID responses stay in order.
SLVERR vs DECERR?	SLVERR from the slave; DECERR from interconnect when no slave decodes.
Who can deadlock and how?	Source waiting for READY before VALID — forbidden.
AXI vs AHB in one line?	AHB: one shared pipelined bus; AXI: split channels, outstanding, OoO.
Bytes in LEN=7, SIZE=3?	8 beats × 8 B = 64 B.